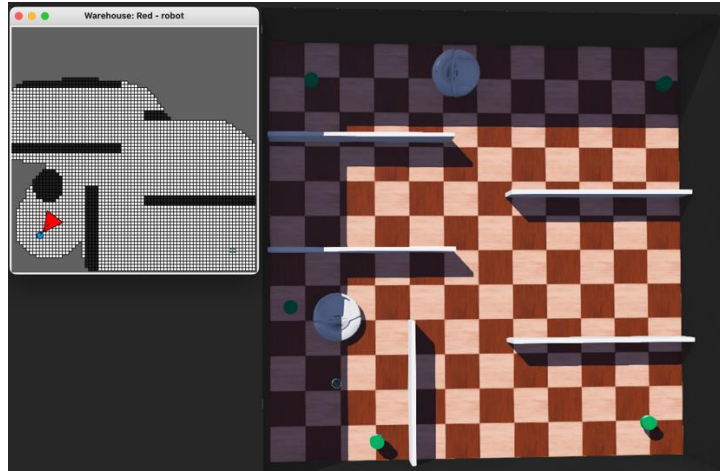


LAB 6: WAREHOUSE AUTOMATION

Due: Wednesday, April 16th 11:59 pm EST

Overview



In this lab, your robot will supervise a warehouse, mapping its environment and avoiding both static (walls) and dynamic (disrupter robots) obstacles. If it hits an obstacle, the run ends. The robot knows its exact location on the map, so localization is not a concern. The goal is to visit all of the five landmarks within 6000 time steps (384 seconds in simulation time). All the important files are present under the folder *controllers/exploration_controller/*. The functions you need to implement are in *controllers/exploration_controller/exploration.py* file.

Detailed descriptions of the code implementation

Your task is to program a robot to explore an unknown warehouse, collect markers, and map the area using its sensors. The robot starts with no knowledge of the warehouse layout and can only detect the regions it explored.

Here is what you will be implementing:

1. **Frontier Planning:** Identify the boundary (frontiers) between explored and unexplored areas. Choose a frontier to explore.
2. **Exploration State Machine:** Calls the frontier planning function to select a frontier and then uses Rapidly-exploring Random Tree (RRT) to plan a path toward that frontier.

The robot explores by selecting a frontier, planning a path, moving to the first waypoint, and then updating its frontiers. This process repeats until all markers are collected.

You will implement the *frontier_planning()* as well as the *exploration_state_machine()* functions in *controllers/exploration_controller/exploration.py* in order to traverse the map and visit all of the landmarks. Your frontiers are the unexplored cells in the map that are adjacent to the explored cells, and your algorithm should move to the centroid of the frontiers.

Frontier Planning

You may find these helper functions useful for implementing *frontier_planning()*:

Grid.is_free(), *Grid.is_in()*

Robot_Sim.explored_cells

get_neighbors() [*exploration.py*]

find_centroid() [*utils.py*]

grid_distance() [*utils.py*]

Algorithm 1 Frontier Planning

```
1: function FRONTIER_PLANNING(robby, grid)
2:   Build a list of frontier cells 1
3:   Group frontier cells (Using connected components labeling)
4:   Calculate centroids of each frontier group
5:   Sort centroids by distance from robot and frontier size
6:   Select best centroid based on distance and accessibility 2
7:   if valid centroid found then
8:     Set robot's next destination to the selected centroid
9:   else
10:    Set robot's next destination to a random frontier cell
11:   end if
12:   return updated robot and next destination
13: end function
```

1. free, explored cells adjacent to unexplored ones.

2. $util = \alpha_1 \cdot distance + \alpha_2 \cdot frontier_length$

Exploration State Machine

In the *exploration_state_machine* function, you are expected to implement the robot's exploration logic. This involves sensing free cells in its field of view, planning a next target coordinate using frontier planning, handling collisions with obstacles by using RRT to replan paths, and setting the robot's wheel velocities to move toward the target.

Note: Using 'grid.markers' will lead to zero points on GraderScope.

As Project 6 will be released prior to the final due date of Project 5, and in Project 5 you are implementing RRT, you will copy your implementation of RRT to use when navigating to your computed frontiers (*grid.py*). However, you will not be submitting this file – it will be only to test locally. To ensure that all students can still get started on Project 6, we provide the RRT implementation on Gradescope for grading. Once Project 5 is **closed for everyone, taking late submissions into account**, we will provide you with this RRT implementation for local testing as well.

Local Testing

We provide 3 maps for your local testing under *controllers/exploration_controller/maps* that can be used for local testing. You can visualize the code locally, by running *exploration_controller/robot_gui.py* with the desired map specified. While there are 5 markers in each map, this file will list the total number of markers as greater than 5 due to the discretization of the world, as you can observe via the GUI. Note that each map is still worth a total of 10 points.

We also provide *autograder.py* file that you can run for local grading. Example usage of *autograder.py*: “python3 autograder.py maps/maze1.json”.

Make sure to take advantage of functions in *utils.py* and remember that your run for either phase will terminate if you collide with an obstacle.

Grading

Rubric	
Found all five markers in the map	10 pts/map

Autograder will grade your exploration on 10 maps.

Submission Materials

Submit your *exploration.py* file and *honor_code.docx* to gradescope.

Running with Webots

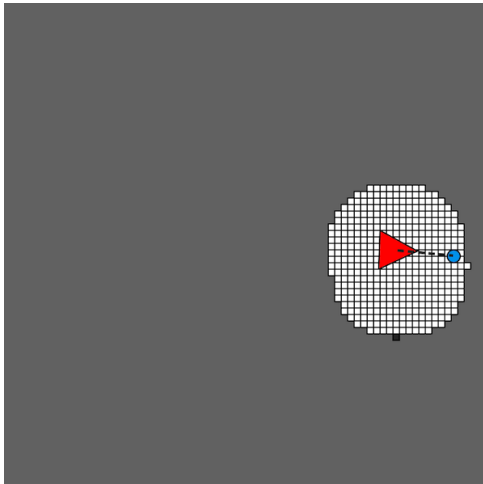
Once the *exploration.py* is complete, you can validate that your algorithm works with the WeBots simulator.

1. Open *exploration_controller.py* and **ensure that the `maze_name` (line: 343) is set to the map you would like to test on**. The map names are “maze1”, “maze2”, and “maze3”. A working implementation should work on all three maps.
2. Launch WeBots. Click File > Open World, then navigate to the “worlds” subfolder. Open the *.wbt* file which matches the `maze_name` from step 1.

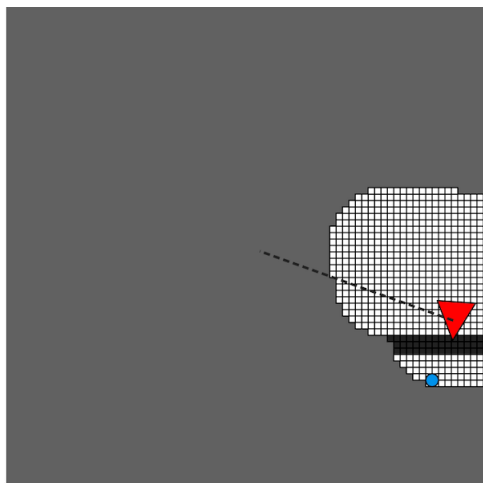
3. A GUI window should appear which shows your exploration code running in a simple 2D approximation of the world to find a path. Then, the robot in WeBots should start moving along this path.
 - a. We recommend selecting the “DEF e-puck Robot” object via the sidebar in the top-left of the WeBots screen. This will allow you to see the coordinate frame of the robot while it moves through the maze.
 - b. Note that the 2D GUI window should automatically close after the robot is done moving.
 - c. Also, the webots execution takes significantly longer time to run compared to GUI and autograder version.

Examples (Robot-GUI)

Robots should navigate towards the frontier



Robot can use RRT to get around the obstacle



Blue dot signifies the goal point

Dotted line signifies the path to the next waypoint in the RRT path

Clarifications

These are a couple of clarifications for Project-6 to answer some of the common doubts:

- **No vl, vr attribute in Robot_Sim:** We expect you to set robbie.vl and robbie.vr in the exploration_state_machine function.
- **How does explore_state_machine() work?** This function is executed by the update function of the environment runner and, exploration_state_machine is executed at a very high frequency. Hence, do not expect the robot to reach the target coordinate within a single function call and you only need to replan if the previous planned target is reached.
- **Debugging:** Please use robot_gui.py file to debug and validate the code. Also, printing the current robot's position and next_coordinate can be helpful too.
- **Robot not moving:** This is probably happening because of some issue in Frontier calculation. A couple of things to keep in mind for Frontier planning:
 - While collecting the initial set of frontier cells, make sure the explored cell you are using for getting the neighbor is free.
 - Sometimes you can have duplicate cells in your initial frontier cells which can offset the centroid calculation. Make sure to not have duplicates in the frontier cells before separating them into frontiers
 - Picking a centroid a bit farther from the robot would be helpful in moving the robot faster.
- **Movement failed: grid (0, 38) isn't free error:** These means your robot collided with the wall.
 - Make sure you are calling RRT properly. Your current position to your next coordinate should be collision free.
 - PID tuning: There is some stochasticity involved. A better tuned PID controller reduces the probability of collision. The better tuned PID the more likely you are to pass all maps.

NOTE: Please do not change the dist parameter in robbie.get_free_cells_in_fov(). We will mark the submission as **INVALID**.

YOU ARE ALSO WELCOME TO ATTEND OUR OFFICE HOURS IN CASE OF PERSISTING DOUBTS!